Fra sensor til alarm

Fugtovervågning hos IIS – en simpel REST API-løsning (delopgave 2)

Gruppe 9 · Brian Jegindø · Mie Mosberg · Nicklas Vilmann · Rene Dalby · Tobias Stryhn

Problemet: fugt giver rust, før noget går i stykker

IIS (Intelligent IoT Solutions) vil overvåge kundernes anlæg døgnet rundt fra et kontrolcenter. Det hedder Managed Services, og det var vores kerneområde i delopgave 1.

Eksempel: fugt i et styreskab i en vindmølle kan give rust og kortslutning. Ser man det i tide, undgår kunden et uventet stop.

Opgaven kræver kæden:

hændelse → beslutning → handling



Hændelse

Pi'en måler fugten og sender den (Sensor Value Received)



Beslutning

Serveren tjekker grænsen: grøn, gul eller rød



Handling

Over 60 %RH oprettes en alarm (Anomaly Detected), som en medarbejder kvitterer for

Det har vi bygget: fire dele, én kæde

Raspberry Pi + Sense HAT

Måler fugt hvert 5. sekund og sender den. LED'en lyser grøn, gul eller rød.

`pi/pi.py`

REST API (FastAPI)

Fire endpoints. Tager imod målinger og bestemmer, om der skal være alarm.

`main.py + rules.py`

PostgreSQL-database

Gemmer alle målinger og alarmer i sin egen Docker-container.

`db.py`

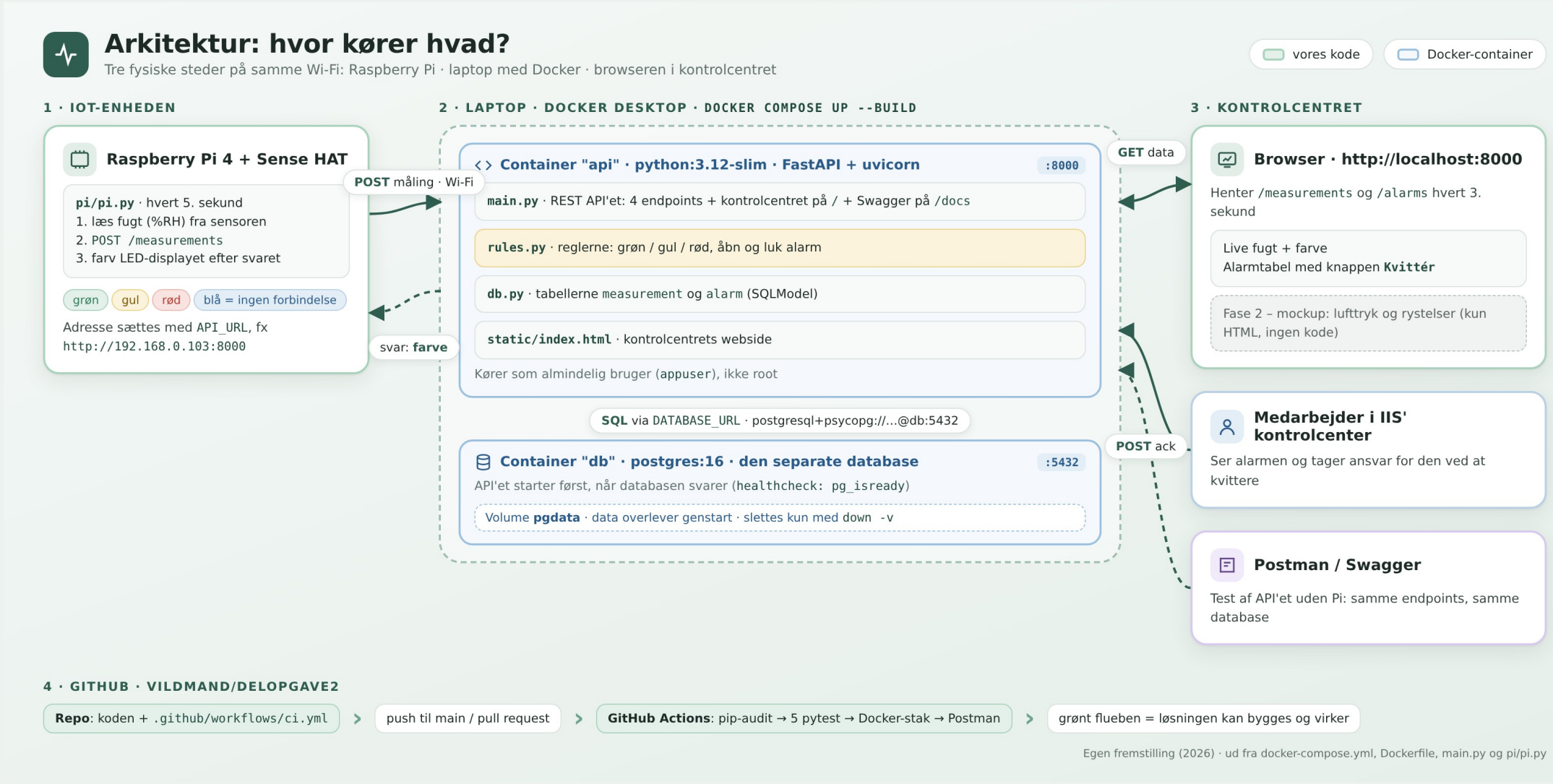
Kontrolcenter

Webside med live fugt, alarmtabel og en Kvittér-knap.

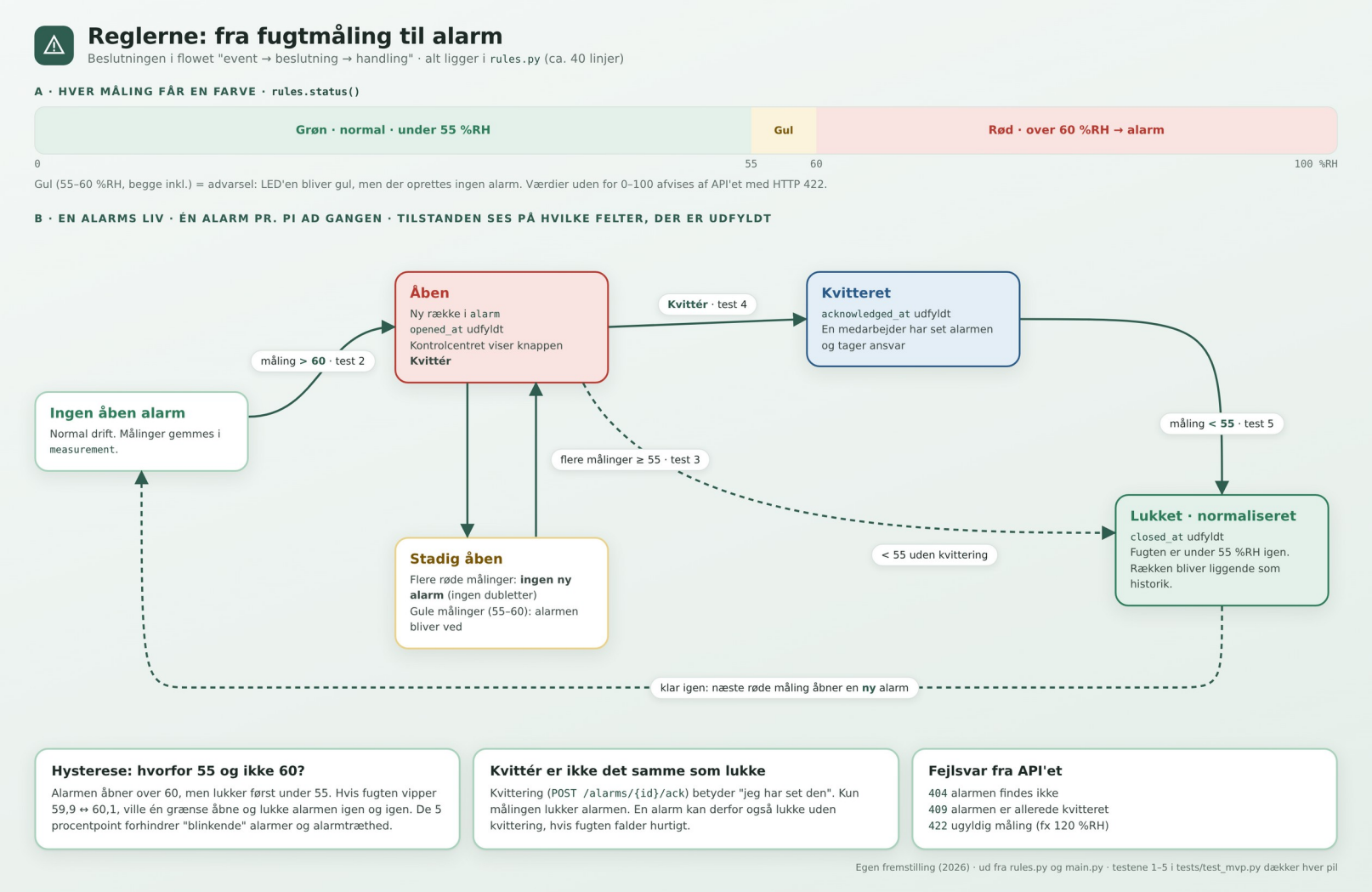
`static/index.html`

Alt kører med én kommando: docker compose up --build. GitHub tester det hele automatisk ved hvert push.

Hvor kører hvad?



Reglerne og en alarms liv



Grøn under 55 %RH

Gul 55–60 %RH

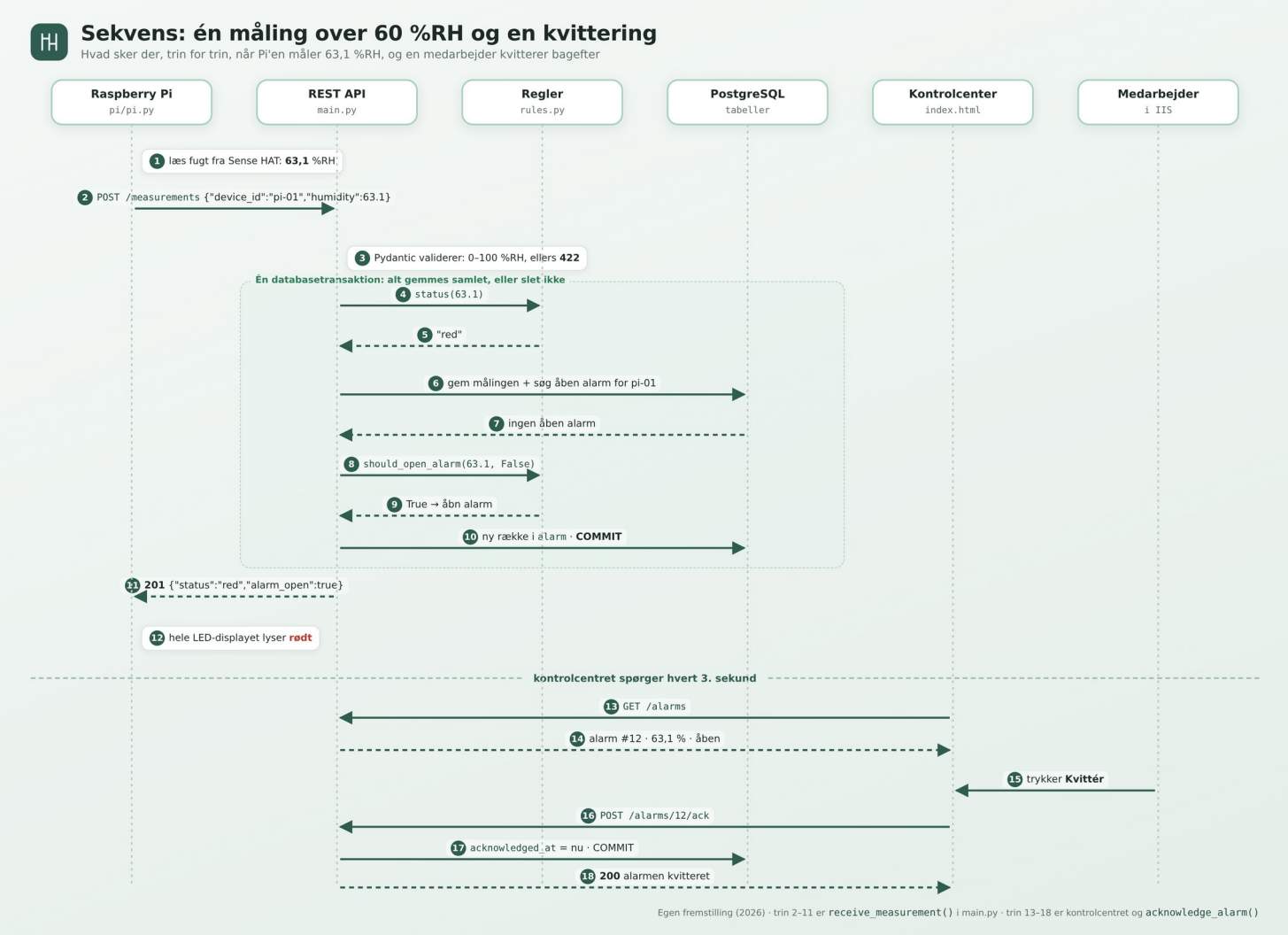
Rød over 60 = alarm

Alarmen starter over 60, men slukker først under 55. Det kaldes hysterese, ligesom en termostat, så alarmen ikke blinker.

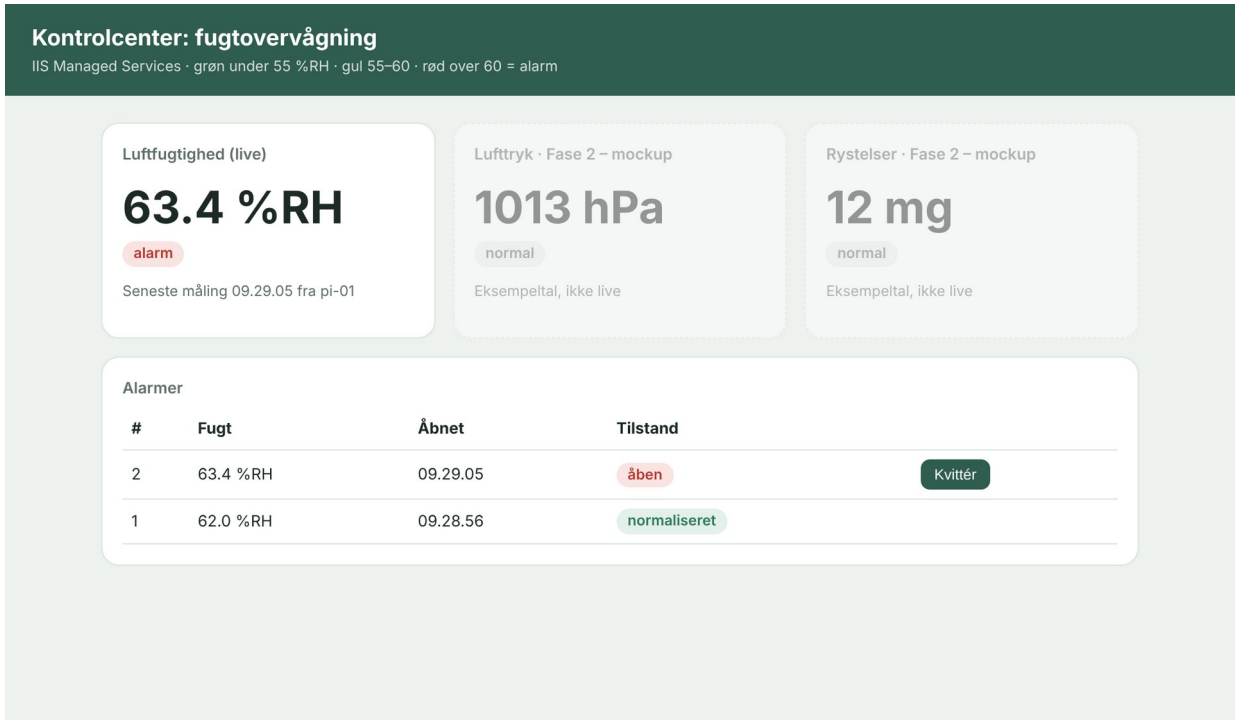
Hver regel har præcis én test. Numrene på pilene er testene.

Én måling fra start til slut

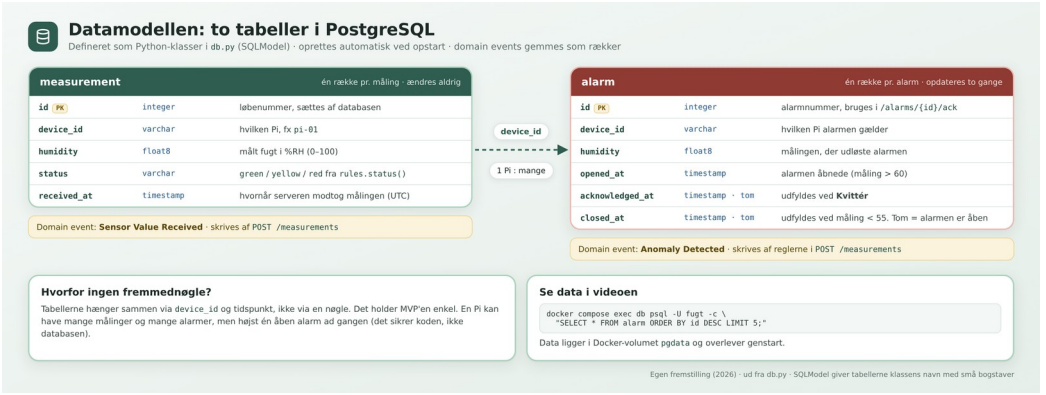
Læs diagrammet oppefra og ned: Pi'en sender, API'et gemmer og tjekker reglen, databasen får en alarm, og svaret farver LED'en.



Det ser medarbejderen – og det bliver gemt



Kontrolcentret: live fugt, alarmtabel og Kvittér. Lufttryk og rystelser er grå mockups (fase 2).



Databasen har kun to tabeller: measurement (alle målinger) og alarm (åbnet, kvitteret, lukket).

Testet automatisk ved hvert push

CI/CD med GitHub Actions

`.github/workflows/ci.yml` · ét job på en frisk Ubuntu-maskine hos GitHub · max 10 minutter · må kun læse koden

HVORNÅR KØRER DEN?

push til `main` | **pull request** | **Run workflow** i hånden (Actions-fanen) › `actions/checkout` + Python 3.12 + `pip install`

FIRE TRIN · STOPPER VED FØRSTE FEJL

1 Sikkerhed
`pip-audit -r requirements.txt`
Har nogen af vores pakker (FastAPI, SQLModel, psycopg …) en kendt sårbarhed?
DevSecOps: sikkerhed tidligt

→

2 Test
`pytest -v`
De 5 tests af reglerne og API'et. Kører mod SQLite i RAM, så de er hurtige.
regel → test, fra strategien

→

3 Byg og start
`docker compose up -d --build`
Bygger API-imaget og starter API + PostgreSQL, præcis som på laptoppen. Venter, til `/alarms` svarer.
"CD": løsningen deployes i et testmiljø

→

4 Postman
`npx newman run postman/…json`
5 kald mod den kørende stak: måling → alarm → kvittering → normalisering. Rammer den rigtige PostgreSQL.
end-to-end, som i videoen

→

✓ Grønt flueben
Commit'en kan bygges, testene består, og API + database virker sammen.
Rødt kryds? Trinnet "Vis API-loggen" printer `docker compose logs api`, så fejlen kan findes.

Hvad betyder CI og CD her?
CI (Continuous Integration): hver ændring bliver automatisk tjekket (trin 1, 2 og 4).
CD (Continuous Delivery): hver ændring bliver bygget til et Docker-image og startet (trin 3). Selve driften kører på laptoppen med `docker compose up --build`.

Vejen frem (ikke bygget i MVP'en)
Den store udgave (RaspPI) har også gitleaks, ruff, bandit og Trivy-scan af imaget, samt et deploy-job via en self-hosted runner på den maskine, der kører løsningen.

Egen fremstilling (2026) · ud fra .github/workflows/ci.yml

5/5	pytest-tests grønne
5/5	Postman-kald grønne
< 1 min	hele CI-kørslen

Hvorfor så simpelt? Vi skar bevidst ned

Første udgave (RaspPI-repo)

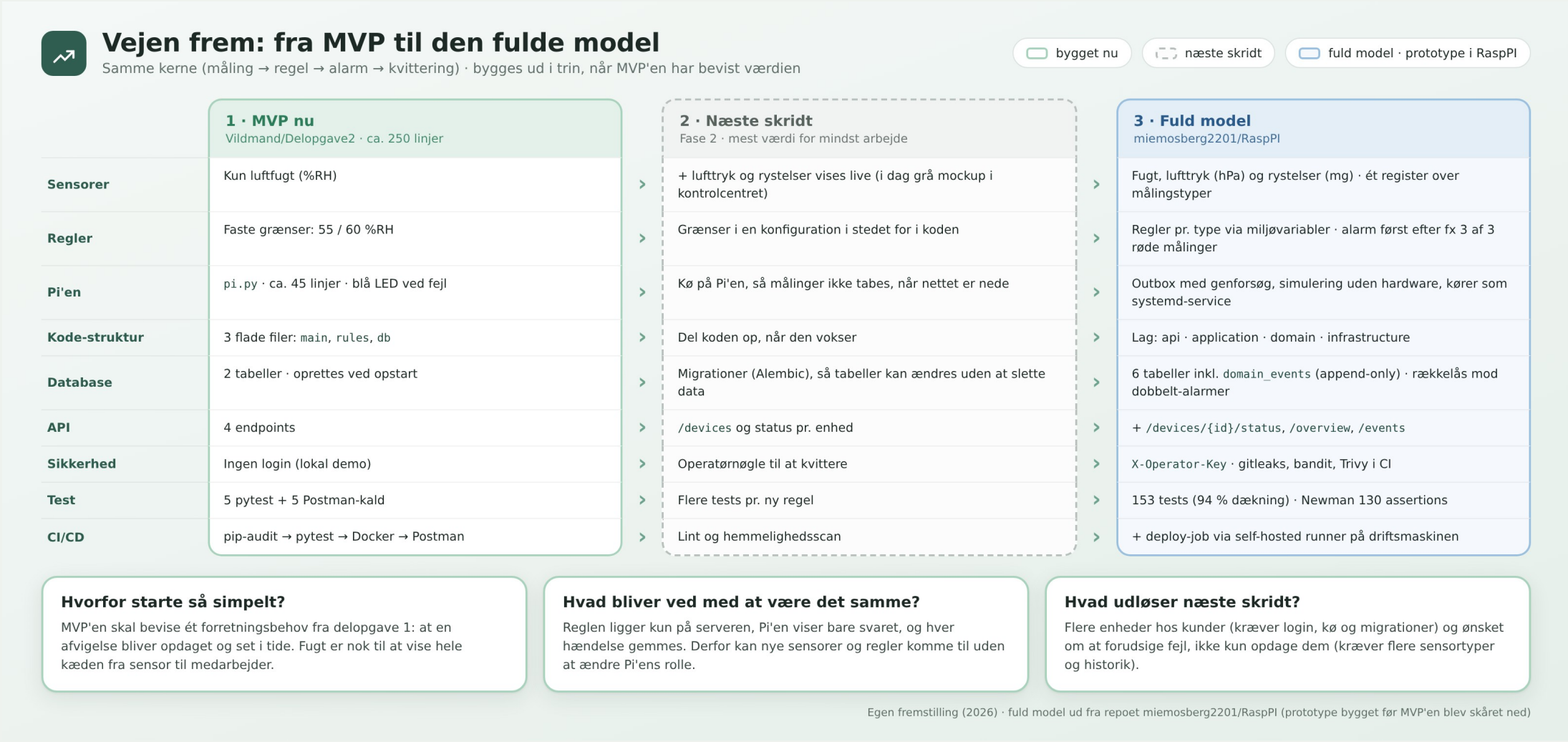
- Tre måletyper: fugt, lufttryk, rystelser
- Hændelseslog, lag, migrationer, outbox
- 153 tests – virkede, men svær at forklare på få minutter

Simpel MVP (Delopgave2-repo)

- Kun fugt live – resten er grå mockups
- Fem filer, som alle i gruppen kan forklare
- 5 tests: strategi → regel → test
- Alle opgavens krav er stadig opfyldt (se KRAV.md)

MVP = den mindste udgave, der virker og kan afprøves. Den store udgave er ikke spildt: den er vores vejen frem.

Vejen frem: skridt for skridt



Prøv det selv

1 · Start serveren

```
docker compose up --build

Åbn bagefter:
localhost:8000
localhost:8000/docs
```

2 · Start Pi'en

```
Kør på Pi'en:
API_URL=http://<PC-IP>:8000
python3 pi.py

Se docs/PI-GUIDE.md
```

3 · Læs mere

```
docs/START-HER.md
docs/KODEGUIDE.md
docs/KRAV.md

github.com/Vildmand/
Delopgave2
```

Pust på Sense HAT'en: LED'en bliver rød, og alarmen dukker op i kontrolcentret.